Name: U.Ganesh **Roll No:** 24501A4459

Branch: CSE (Data Science) **Date:**

Mini Project: Predicting Air Quality (Carbon Monoxide Concentration) Using the CRISP-DM Methodology

Aim: Development of a robust end-to-end machine learning pipeline adhering to the Cross-Industry Standard Process for Data Mining (CRISP-DM) methodology.

Project Title: Predicting Hourly CO Concentration Using Multi-Sensor Readings, Pollutant Levels, Weather and Time Features

1. Business Understanding

1.1 Problem Statement & Relevance

Air quality monitoring is important for public health and city planning. Carbon monoxide (CO), released mainly by road traffic, is one of the key pollutants that authorities track. Reference analysers measure CO accurately, but they are expensive, need regular calibration and can have gaps in their records. In this dataset about 18% of the hourly CO readings are missing. Low-cost metal-oxide gas sensors keep recording, but their responses are indirect and are affected by other gases, temperature and humidity.

This project formulates the problem as a supervised **regression** task. The target variable is **CO(GT)**, the true hourly averaged CO concentration in mg/m^3. The inputs are the other sensor responses, pollutant concentrations, temperature, humidity and time features (hour, month, day of week). The work follows the CRISP-DM phases: Business Understanding, Data Understanding, Data Preparation, Modelling, Evaluation and Deployment.

1.2 Expected Benefits

- Missing or unavailable CO readings can be estimated from the other sensors, keeping the monitoring record continuous.
- The sensors and pollutants that influence CO most can be identified.
- Hours and months with high predicted CO can support early warnings and traffic or pollution-control planning.
- MAE, MSE and RMSE show the size of the prediction error in the original CO units (mg/m^3).
- R^2 shows how much of the variation in CO the model explains.
- Time-ordered validation gives a realistic estimate of performance on future, unseen data.

2. Data Understanding

2.1 Dataset Structure

The dataset file `Copy of AirQualityUCI.csv` (the UCI Air Quality dataset) has **9,471 rows and 17 columns** as loaded. Each row is one hourly averaged reading from a multi-sensor air quality device, starting on 10-03-2004 at 18:00. It contains five metal-oxide sensor responses (PT08.S1 to PT08.S5), reference analyser readings for CO, NMHC, benzene, NOx and NO2, and temperature, relative humidity and absolute humidity. Two columns (Unnamed: 15 and Unnamed: 16) and 114 rows are completely empty, so after removing them the cleaned dataset has **9,357 rows and 15 columns**.

2.2 Data Dictionary

Variable	Description
Date	Date of the reading (dd-mm-yyyy)
Time	Hour of the reading (hh.mm.ss)
CO(GT)	True hourly averaged CO concentration (mg/m^3) – target
PT08.S1(CO)	Tin oxide sensor response (nominally targeted at CO)
NMHC(GT)	True hourly averaged non-methane hydrocarbons ($\mu g/m^3$)
C6H6(GT)	True hourly averaged benzene ($\mu g/m^3$)
PT08.S2(NMHC)	Titania sensor response (nominally targeted at NMHC)
NOx(GT)	True hourly averaged NOx (ppb)
PT08.S3(NOx)	Tungsten oxide sensor response (nominally targeted at NOx)
NO2(GT)	True hourly averaged NO2 ($\mu g/m^3$)
PT08.S4(NO2)	Tungsten oxide sensor response (nominally targeted at NO2)
PT08.S5(O3)	Indium oxide sensor response (nominally targeted at O3)
T	Temperature (°C)
RH	Relative humidity (%)
AH	Absolute humidity
Unnamed: 15, 16	Completely empty columns (removed)

2.3 Data Quality Observations

- The dataset loads as 9,471 rows and 17 columns: 15 float and 2 text columns (Date, Time). Two columns and 114 rows are entirely empty and were removed, leaving 9,357 rows and 15 columns.
- No duplicate rows were found.
- Missing readings are stored as **-200** in the file. These were replaced with NaN before analysis.
- CO(GT), the target, is missing in **17.99%** of rows (1,683 rows). NOx(GT) and NO2(GT) are missing in about 17.5% of rows, and every sensor, temperature and humidity column in 3.91%.
- NMHC(GT) is missing in **90.23%** of rows (only 914 valid values), so it is not usable as a feature.
- After removing rows without a target value, **7,674 rows** remain for modelling.
- The target is right-skewed: mean 2.15 mg/m^3, median 1.8, maximum 11.9.

Missing values and statistical summary (after replacing -200 with NaN)

Feature	Non-missing	Missing %	Mean	Std	Min	Max
CO(GT)	7,674	17.99	2.15	1.45	0.10	11.90
PT08.S1(CO)	8,991	3.91	1099.83	217.08	647	2040
NMHC(GT)	914	90.23	218.81	204.46	7	1189
C6H6(GT)	8,991	3.91	10.08	7.45	0.10	63.70
PT08.S2(NMHC)	8,991	3.91	939.15	266.83	383	2214
NOx(GT)	7,718	17.52	246.90	212.98	2	1479
PT08.S3(NOx)	8,991	3.91	835.49	256.82	322	2683
NO2(GT)	7,715	17.55	113.09	48.37	2	340
PT08.S4(NO2)	8,991	3.91	1456.26	346.21	551	2775
PT08.S5(O3)	8,991	3.91	1022.91	398.48	221	2523
T	8,991	3.91	18.32	8.83	-1.90	44.60
RH	8,991	3.91	49.23	17.32	9.20	88.70
AH	8,991	3.91	1.03	0.40	0.18	2.23

Python code: importing libraries, loading and cleaning the dataset

```
import numpy as np, pandas as pd import matplotlib.pyplot as plt, seaborn as
sns, joblib from sklearn.model_selection import (train_test_split,
GridSearchCV,                                    cross_val_score,
TimeSeriesSplit) from sklearn.preprocessing import StandardScaler from
sklearn.impute import SimpleImputer from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score from sklearn.linear_model import LinearRegression, Ridge from
sklearn.tree import DecisionTreeRegressor from sklearn.ensemble import
RandomForestRegressor from sklearn.neighbors import KNeighborsRegressor from
sklearn.svm import SVR

df = pd.read_csv("/content/Copy of
AirQualityUCI.csv") print("Shape:", df.shape)
df.info() display(df.head())

df = df.dropna(axis=1, how="all").dropna(how="all") print("Shape
after removing empty rows/cols:", df.shape) print("Duplicates:",
df.duplicated().sum()) df = df.replace(-200, np.nan)
# -200 = missing print((df.isnull().mean()*100).round(2))      # %
missing per column display(df.describe())
```

Output

```
Shape: (9471, 17)
Shape after removing empty rows/cols: (9357, 15)
Duplicates: 0
Missing %: CO(GT) 17.99 | NMHC(GT) 90.23 | NOx(GT) 17.52 | NO2(GT) 17.55
all other numeric columns 3.91 | Date, Time 0.00
```

2.4 Exploratory Data Analysis (EDA)

The notebook plots the distribution of the target CO(GT), a correlation heatmap of all numeric columns, boxplots of every numeric feature, and a scatter plot of benzene C6H6(GT) against CO(GT).

```
sns.histplot(df["CO(GT)"].dropna(), kde=True); plt.show() plt.figure(figsize=(10,8))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm", fmt=".2f");
plt.show() df.drop(columns=["Date","Time"]).boxplot(figsize=(12,5), rot=90); plt.show()
sns.scatterplot(x="C6H6(GT)", y="CO(GT)", data=df); plt.show()
```

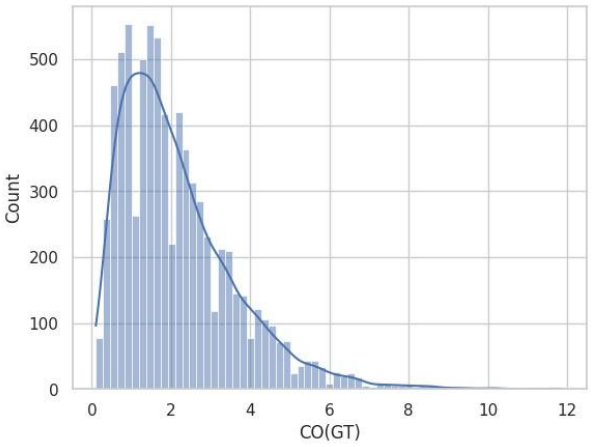


Figure 1: Distribution of CO(GT)

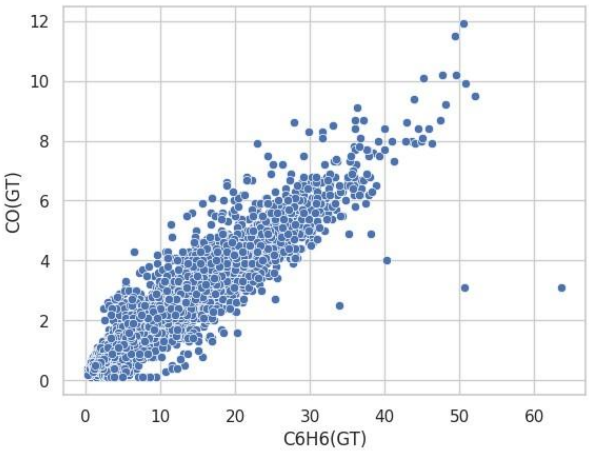


Figure 2: C6H6(GT) versus CO(GT)

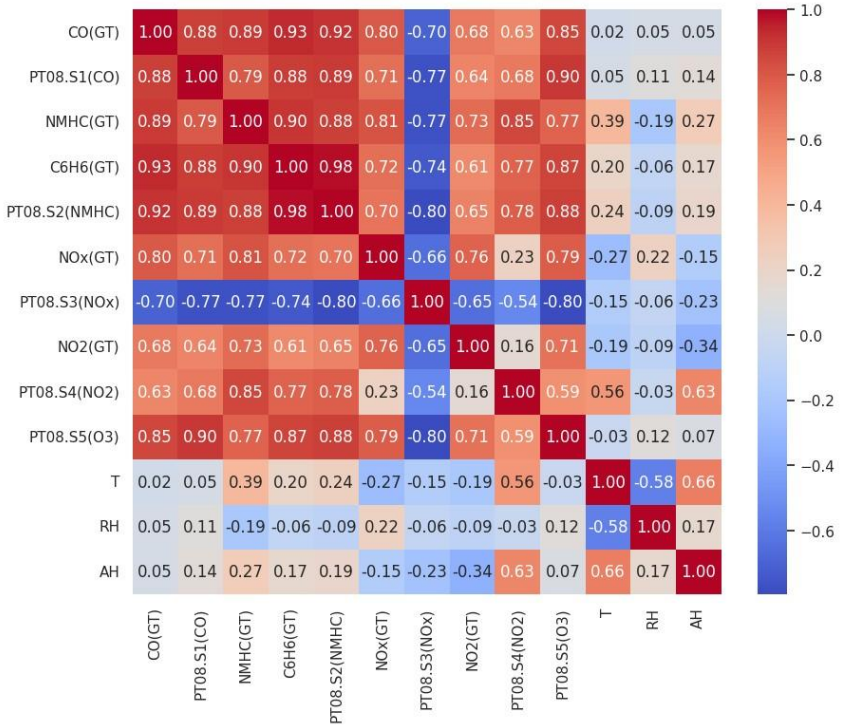


Figure 3: Correlation heatmap of the numeric columns

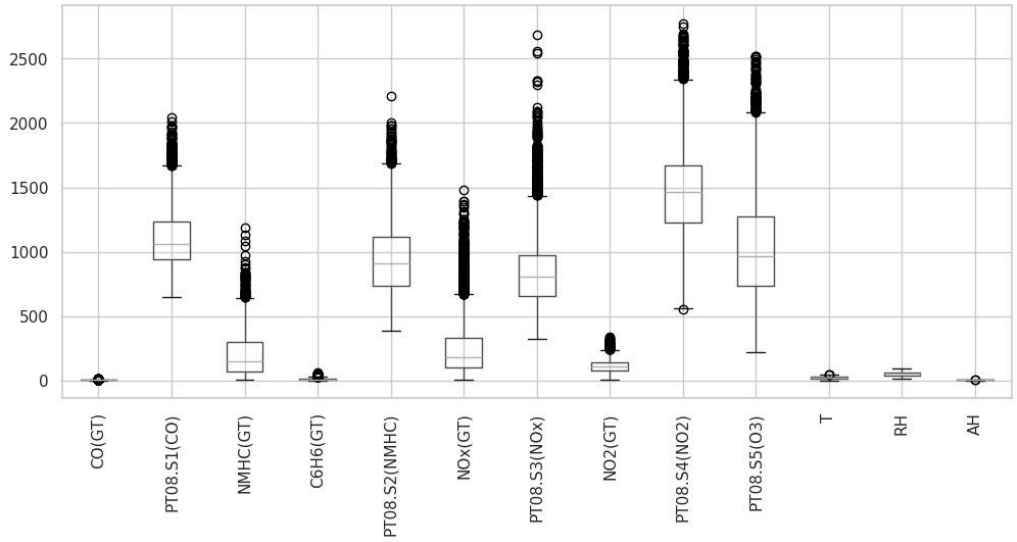


Figure 4: Boxplots of the numeric features

Correlation of each feature with CO(GT)

Feature	Correlation	Feature	Correlation
C6H6(GT)	0.93	PT08.S3(NOx)	-0.70
PT08.S2(NMHC)	0.92	NO2(GT)	0.68
NMHC(GT)	0.89	PT08.S4(NO2)	0.63
PT08.S1(CO)	0.88	RH	0.05
PT08.S5(O3)	0.85	AH	0.05
NOx(GT)	0.80	T	0.02

EDA findings

- CO(GT) is right-skewed: the mean (2.15) is above the median (1.8) and the maximum reaches 11.9 mg/m^3.
- Benzene C6H6(GT) (0.93) and the titania sensor PT08.S2(NMHC) (0.92) have the strongest positive correlation with CO. PT08.S1(CO), PT08.S5(O3) and NOx(GT) are also strongly related (0.80 to 0.88).
- PT08.S3(NOx) is the only strongly negative feature (-0.70).
- The sensors are strongly correlated with each other (for example C6H6(GT) and PT08.S2(NMHC) = 0.98), so multicollinearity is present.
- Temperature, relative humidity and absolute humidity have almost no linear correlation with CO (0.02 to 0.05).
- The pollutant columns have large gaps between the 75th percentile and the maximum (for example NOx(GT): 326 against 1479), which indicates high-value outliers.
- NMHC(GT) correlates well with CO (0.89), but with 90.23% of its values missing it cannot be used.

3. Data Preparation

3.1 Feature Engineering and Removing Unusable Columns

Date and Time were combined into one datetime value, from which three time features were created: **hour**, **month** and **day of week**. The original Date, Time and Datetime columns were then dropped. NMHC(GT) was also dropped because about 90% of its values are missing.

```
df["Datetime"] = pd.to_datetime(df["Date"]+" "+df["Time"], format="%d-%m-%Y %H.%M.%S")
df["hour"] = df["Datetime"].dt.hour
df["month"] = df["Datetime"].dt.month
df["dayofweek"] = df["Datetime"].dt.dayofweek
df = df.drop(columns=["Date","Time","Datetime","NMHC(GT)"])   # NMHC: ~90% missing
```

3.2 Separating Features and Target

Rows where the target CO(GT) is missing cannot be used for supervised learning, so they were removed. Missing values in the feature columns are handled later inside the pipeline.

```
df = df.dropna(subset=["CO(GT)"])
X = df.drop("CO(GT)", axis=1)
y = df["CO(GT)"]
print(X.shape, y.shape)
```

Output: (7674, 14) (7674,)

The 14 input features are: PT08.S1(CO), C6H6(GT), PT08.S2(NMHC), NOx(GT), PT08.S3(NOx), NO2(GT), PT08.S4(NO2), PT08.S5(O3), T, RH, AH, hour, month and dayofweek. All of them are numeric, so no categorical encoding is needed.

3.3 Train/Test Split (Chronological)

The data is hourly time-series data, so it was split 80:20 **without shuffling**. The first 80% of the records (the earlier period) are used for training and the last 20% (the later period) for testing. This avoids using future information to predict the past and mimics real forecasting use.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, shuffle=False)
print(X_train.shape, X_test.shape)
```

Output: (6139, 14) (1535, 14)

3.4 Preprocessing Pipeline

Missing feature values are imputed with the median and all features are standardised. The preprocessing step is part of every model pipeline, so the imputation values and scaling statistics are learned only from the training data (or the training part of each cross-validation fold).

```
preprocessor = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())])
```

4. Modelling

4.1 Model Approach

The target is a continuous value, so six regression algorithms were compared:

- **Linear Regression:** fits a straight-line relationship; a simple, interpretable baseline.
- **Ridge Regression:** linear regression with L2 regularisation (alpha = 1), useful when features are correlated.
- **Decision Tree:** learns if-then splits on the features.
- **Random Forest:** combines 200 decision trees for stability.

- **KNN:** predicts from the average of the 5 most similar hours.
- **Support Vector Regression (SVR):** fits a margin-based non-linear function (default RBF kernel).

Each model is placed inside a scikit-learn `Pipeline` together with the preprocessor, so identical transformations are applied to training and test data. Cross-validation uses `TimeSeriesSplit` with 5 splits, so every validation fold lies after its training data in time.

4.2 Training, Prediction and Cross-Validation

```
models = {
    "Linear Regression": LinearRegression(),
    "Ridge": Ridge(alpha=1.0),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(n_estimators=200, random_state=42),
    "KNN": KNeighborsRegressor(n_neighbors=5),
    "SVR": SVR()}

results = [] tscv =
TimeSeriesSplit(n_splits=5) for
name, model in models.items():
    pipe = Pipeline([("prep", preprocessor), ("model", model)])
pipe.fit(X_train, y_train)     pred = pipe.predict(X_test)     cv_r2
= cross_val_score(pipe, X, y, cv=tscv, scoring="r2").mean()
results.append([name,
mean_absolute_error(y_test, pred),
mean_squared_error(y_test, pred),
np.sqrt(mean_squared_error(y_test, pred)),
r2_score(y_test, pred), cv_r2])

res = pd.DataFrame(results,
columns=["Model","MAE","MSE","RMSE","R2","CV_R2"]) res =
res.sort_values("R2", ascending=False) display(res)
```

5. Evaluation

5.1 Metric Selection

Since the target is continuous, regression metrics are used instead of accuracy, precision and recall.

- **MAE** (Mean Absolute Error): average size of the error, in mg/m^3.
- **MSE** (Mean Squared Error): average squared error; penalises large mistakes more.
- **RMSE** (Root Mean Squared Error): square root of MSE, in the same unit as CO.
- **R^2:** share of the variation in CO explained by the model (1.0 is perfect).
- **5-split time-series cross-validation R^2:** a more stable estimate than a single test period.

5.2 Model Comparison

Model	MAE	MSE	RMSE	R2	CV R^2
Linear Regression	0.3463	0.2748	0.5242	0.8480	0.8541
Ridge	0.3467	0.2750	0.5244	0.8479	0.8543
SVR	0.3599	0.3016	0.5492	0.8332	0.6930
Random Forest	0.3773	0.3158	0.5620	0.8253	0.8554
KNN	0.4515	0.4109	0.6410	0.7727	0.8116
Decision Tree	0.5784	0.7269	0.8526	0.5979	0.7384

- Linear Regression and Ridge give almost identical results (test R^2 about 0.848), which fits the strong, nearly linear relationship between CO and the other sensors seen in the EDA.
- Random Forest has the highest cross-validated R^2 (0.8554), so it was taken forward for tuning. Linear Regression and Ridge score slightly higher on the single test period.
- SVR is good on the test period (0.8332) but unstable across validation folds (CV R^2 0.6930).
- The Decision Tree overfits and performs worst (R^2 0.5979).

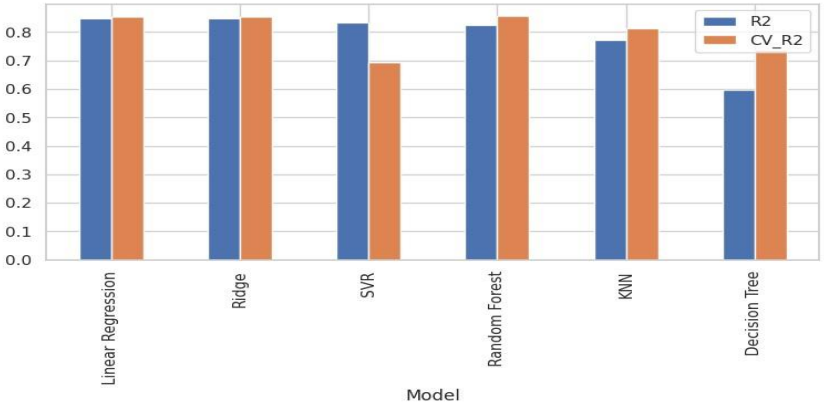


Figure 5: Test R^2 and cross-validation R^2 for each model

5.3 Hyperparameter Tuning

GridSearchCV with 5-split time-series cross-validation tunes the number of trees, the maximum depth and the minimum samples per leaf of the Random Forest (12 parameter combinations).

```
pipe = Pipeline([("prep", preprocessor),
                 ("model", RandomForestRegressor(random_state=42))])
grid = GridSearchCV(pipe,
    {"model__n_estimators": [100, 200],
     "model__max_depth": [None, 10, 20],
"model__min_samples_leaf": [1, 3]},
cv=tscv, scoring="r2", n_jobs=-1)
grid.fit(X_train, y_train)
print(grid.best_params_, grid.best_score_)
best = grid.best_estimator_
```

Output

```
{'model__max_depth': 20, 'model__min_samples_leaf': 3, 'model__n_estimators': 100}
Best cross-validation R2: 0.8666
```

5.4 Final Performance on the Test Set

```
pred = best.predict(X_test) print("MAE :",
mean_absolute_error(y_test, pred)) print("MSE :",
mean_squared_error(y_test, pred)) print("RMSE:",
np.sqrt(mean_squared_error(y_test, pred))) print("R2
:", r2_score(y_test, pred))
```

Metric	Score
MAE	0.3558
MSE	0.2863
RMSE	0.5350
R_2	0.8417

On average the tuned model is off by about 0.36 mg/m^3 (the overall mean CO is about 2.15 mg/m^3), and it explains about 84% of the variation in CO on the unseen later period.

5.5 Prediction Diagnostics

```
plt.scatter(y_test, pred, alpha=0.4); plt.plot([0,10],[0,10],"r--") plt.xlabel("Actual
CO"); plt.ylabel("Predicted CO"); plt.show()   # actual vs predicted
sns.histplot(y_test - pred, kde=True); plt.show()                # residuals
```

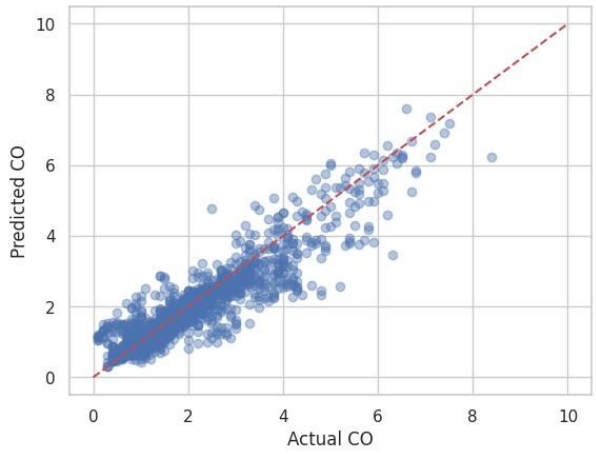


Figure 6: Actual versus predicted CO

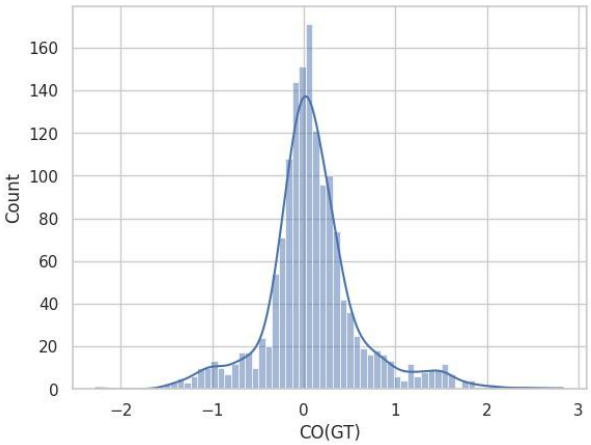


Figure 7: Distribution of residuals

- Predictions follow the diagonal closely for low and medium CO values.
- A few high-CO hours (above about 5 mg/m^3) are under-predicted.
- Residuals are centred close to zero and mostly fall between -1 and +1 mg/m^3.

5.6 Feature Importance

```
imp = pd.Series(best.named_steps["model"].feature_importances_,
index=X.columns).sort_values() imp.plot(kind="barh"); plt.show()
```

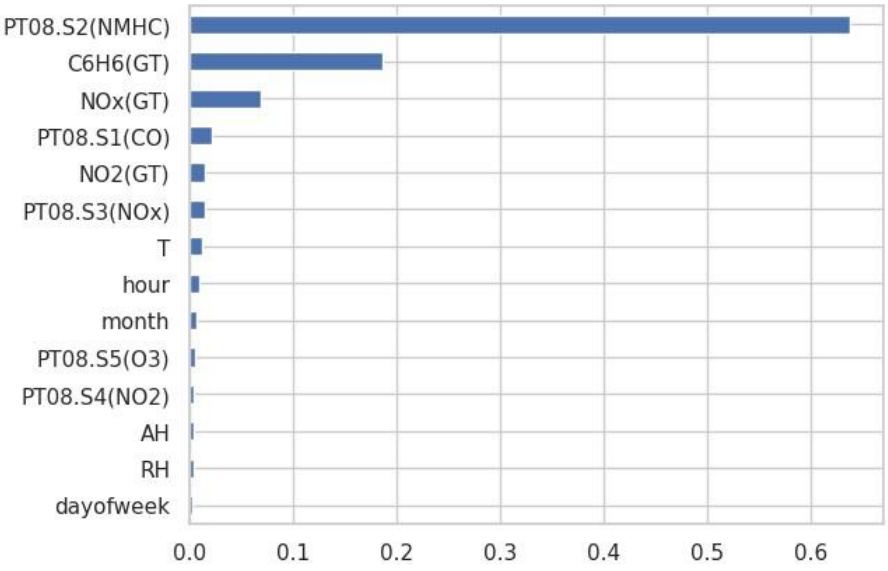


Figure 8: Random Forest feature importance

- PT08.S2(NMHC) is by far the most important feature (about 0.64), followed by C6H6(GT) (about 0.19) and NOx(GT) (about 0.07).
- PT08.S1(CO), NO2(GT), PT08.S3(NOx) and temperature contribute a little; the time features (hour, month, day of week), PT08.S5(O3), PT08.S4(NO2), AH and RH add very little.

Importance values are read from the plot and are approximate.

6. Deployment and Final Project Summary

Preprocessing and model are packaged as one scikit-learn `Pipeline`, so a new raw record can be passed in and scored directly. The pipeline is saved with joblib, reloaded, and tested on raw rows from the test set.

```
joblib.dump(best, "best_air_quality_pipeline.pkl") loaded =
joblib.load("best_air_quality_pipeline.pkl")
print(loaded.predict(X_test.iloc[:3]))      # test on raw
rows
```

Output: [0.56117233 0.69143889 0.7429035]

Item	Value
Records after cleaning / used for modelling	9,357 / 7,674
Input features	14 (11 sensor, pollutant and weather columns + 3 time features)
Training / testing records	6,139 / 1,535 (chronological split)
Models compared	6
Best model	Random Forest (max_depth = 20, min_samples_leaf = 3, n_estimators = 100)
Test MAE / RMSE / R^2	0.3558 / 0.5350 / 0.8417
Cross-validation R^2 (tuned)	0.8666

7. Conclusion

7.1 Summary of Results

An end-to-end machine learning pipeline was developed to predict hourly CO concentration using the CRISP-DM methodology. After cleaning, the dataset had 9,357 hourly records; missing values coded as -200 were converted to NaN, NMHC(GT) was dropped for being 90% missing, and rows without a CO value were removed, leaving 7,674 records. Six regression models were compared with a chronological split and time-series cross-validation. A tuned Random Forest reached a test R^2 of 0.8417, MAE of 0.3558 and RMSE of 0.5350 mg/m^3, with a cross-validation R^2 of 0.8666.

7.2 Key Insights

- Benzene C6H6(GT) and the PT08.S2(NMHC) sensor are the strongest indicators of CO; PT08.S2(NMHC) carries most of the Random Forest's predictive power.
- Temperature and humidity have almost no linear correlation with CO, and the time features add little to the model.
- The sensors are highly inter-correlated, which is why simple linear models perform almost as well as the ensemble.
- A chronological split with TimeSeriesSplit avoids look-ahead leakage and gives a more honest estimate than a random split on time-ordered data.
- A single Pipeline keeps training and test transformations consistent and makes deployment simple.
- The Decision Tree overfits and gives the weakest results.

7.3 Limitations & Future Work

- The tuned Random Forest (test R^2 0.8417) is slightly below the untuned Linear Regression (0.8480) on the single test period, so the gain from the more complex model is small. Choosing the final model by repeated time-based validation would be more robust.
- The test set is one later period of a roughly one-year dataset; seasonal behaviour in months not seen in training may affect the scores, and the month feature may not generalise to such months.
- NOx(GT), NO2(GT) and C6H6(GT) come from reference analysers. If the aim is to replace the CO analyser, these may also be unavailable, so a model using only the low-cost sensors, temperature and humidity should be tested.
- About 18% of the target rows were dropped and the remaining feature gaps were filled with the median; time-based interpolation could be tried instead.
- High CO hours are slightly under-predicted, and the target is right-skewed.
- Further work: add lag features (previous hours' CO and sensor values), try Gradient Boosting or XGBoost and sequence models, and deploy the saved pipeline as a prediction web app.

8. Appendix

8.1 Plots

- Distribution of CO(GT) and C6H6(GT) versus CO(GT) scatter plot (Figures 1 and 2)
- Correlation heatmap and boxplots of the numeric features (Figures 3 and 4)
- Model comparison chart of test R^2 and cross-validation R^2 (Figure 5)
- Actual versus predicted CO and residual distribution (Figures 6 and 7)
- Random Forest feature importance (Figure 8)

8.2 Dataset Summary

Item	Value
Records / columns as loaded	9,471 / 17
Records / columns after removing empty rows and columns	9,357 / 15
Duplicates removed	0
Target	CO(GT), mg/m^3 (7,674 valid values)
Target mean / median / max	2.15 / 1.80 / 11.90
Features used	14
Best model	Random Forest (tuned)

8.3 Python Code Snippets

All code snippets used in this project are reproduced in the relevant sections above.